VDK/TLM Creation

Rapid Training

Training SPI Controller Model Specification

© 2018 Synopsys, Inc.

Table of Contents

Training Example Specification: SPI Controller Model **2**
Base Functionality 2
Structure of the SPI Controller Model **2**
Behavior of the SPI Controller Model **3**

Training Example Specification: SPI Controller Model

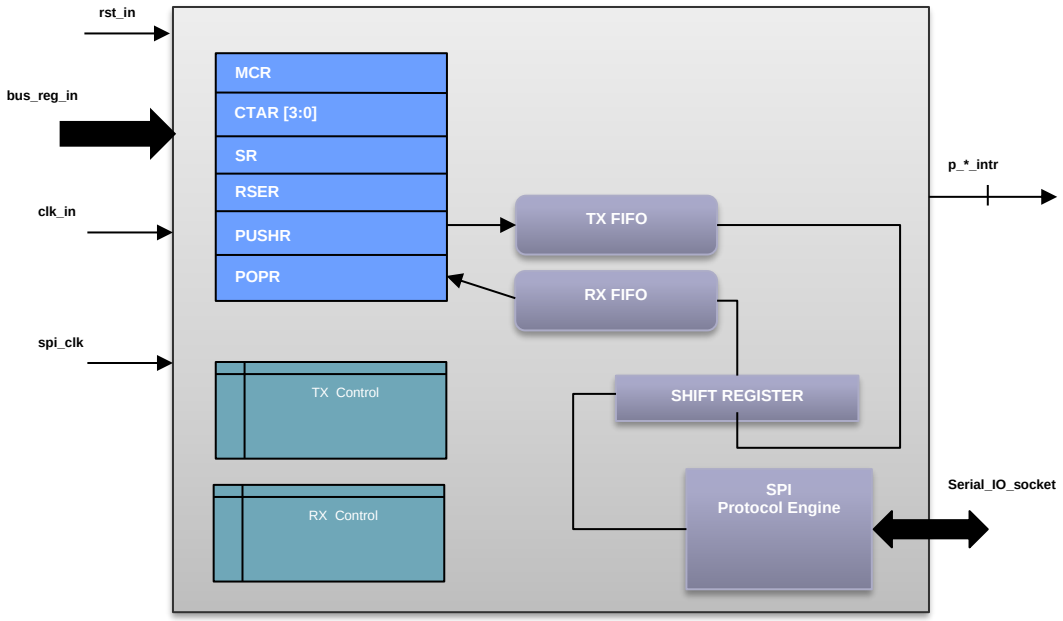
Base Functionality

The example/training SPI controller is a very much simplified version of a Serial Peripheral Interface only.

A ‘real-life’ SPI controller is obviously more complex and contains many more details, but for sake of time we peripheral functionality and provide only the required information. The goal is to teach the (new) tool features and not the peripheral model itself.

Structure of the SPI Controller Model

The structure of the SPI controller is depicted in the following:

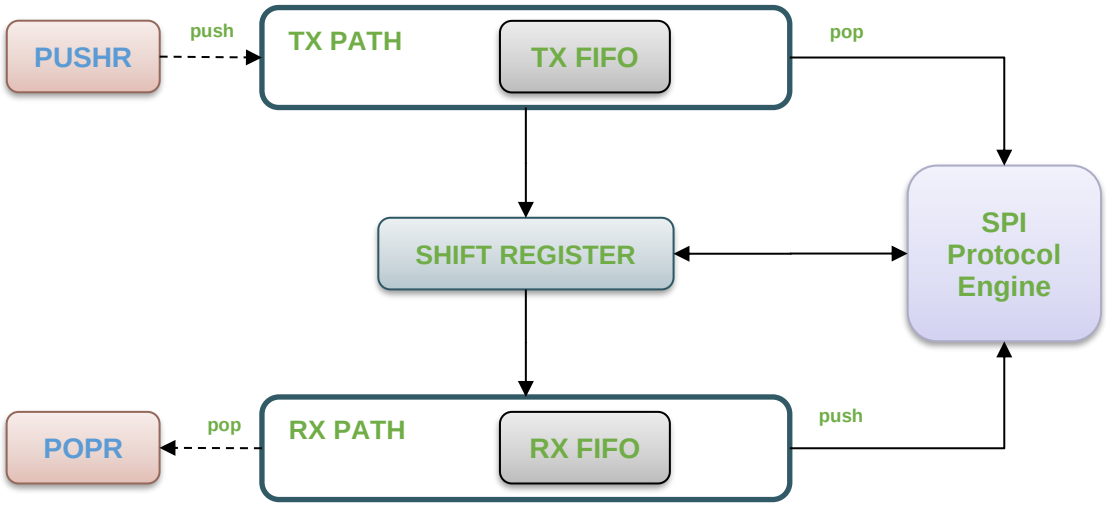


Behavior of the SPI Controller Model

The SPI controller can initiate SPI transfers when configured as master. The data is pushed into the TX FIFO through the PUSHR register. From there goes into the Shift Register and then it is transfer using the SPI interface. SPI is a bidirectional communication protocol and at the same time, new data is transferred from the SPI interface to the Shift Register and to the RX FIFO. The data can be retrieved from the RX FIFO through the POPR register.

When configured as slave, the transfer is initiated by another master in the system. The SPI interface receives the transfer and first pops a value from the TX FIFO and stores it in the Shift Register. This value is sent back simultaneously with the incoming transfer. The incoming value is stored in the Shift Register and then in the RX FIFO. During this operation the status of the controller is updated on the status register SR, and the required interrupts are generated (overflow, underflow, fill and drain conditions).

The functionality of the SPI controller can be described with the block diagram of the figure below:



In **Master mode**, data is pushed to the transmission control logic (TX PATH) via the PUSHR register. The data will be stored in a fifo (TX FIFO) and the transmission control logic will fetch the data from the TX FIFO (when new data is available) and copy it to the Shift Register and start a new transaction on the protocol engine. The SPI Protocol Engine will fetch the data to be sent from the TX PATH when it is ready to send. Since SPI is a bidirectional protocol, simultaneously data will arrive to the SPI Protocol Engine. This received data is copied to the Shift Register by the SPI protocol engine. And from the Shift Register to the RX FIFO by the reception control logic (RX PATH).

In **Slave mode**, data is received by the SPI Protocol Engine. That data is copied into the Shift Register and later to the RX FIFO by the Reception control logic (RX PATH). The RX PATH logic initiates as well a response transfer by reading data from the TX FIFO and then sending it back through the SPI Protocol Engine.